

FHSS Resampling Effect Lab

Ảnh hưởng của resampling đến giấu tin FHSS trong âm thanh

1. Overview

Bài lab fhss_resample_effect minh họa ảnh hưởng của resampling đối với dữ liệu ẩn FHSS trong âm thanh. Người học khôi phục thông điệp từ file stego ban đầu, sau đó dùng sox để đổi sample rate xuống thấp hơn rồi đưa file trở lại sample rate ban đầu. Mục tiêu là quan sát rằng file sau resampling vẫn có thể là WAV hợp lệ, nhưng thông điệp FHSS không còn khôi phục đúng vì các giá trị sample đã bị nội suy và làm tròn lại.

1.1 Cơ sở lý thuyết

FHSS, viết tắt của Frequency Hopping Spread Spectrum, là ý tưởng truyền hoặc nhúng dữ liệu bằng cách thay đổi vị trí theo một chuỗi giả ngẫu nhiên sinh từ khóa. Trong bài lab này, ý tưởng đó được mô phỏng trên file WAV: mỗi bit của thông điệp được biểu diễn bằng một thay đổi rất nhỏ trên một sample âm thanh, nhưng vị trí sample được chọn theo khóa. Nếu dùng đúng khóa và đúng kích thước frame, chương trình khôi phục sẽ đọc đúng chuỗi sample đã nhúng. Nếu dùng sai khóa hoặc dữ liệu sample đã bị biến dạng, thông điệp khôi phục sẽ bị sai.

Resampling là quá trình đổi sample rate của âm thanh, ví dụ từ 44100 Hz xuống 22050 Hz. Khi resample, phần mềm âm thanh không chỉ đổi metadata, mà phải tính lại các sample trên một lưới thời gian mới. Quá trình này thường dùng nội suy, lọc và làm tròn giá trị sample. Vì vậy, ngay cả khi file được resample ngược lại về 44100 Hz, các sample bên trong không còn giống hoàn toàn với file ban đầu.

Đây là vấn đề quan trọng với FHSS dựa trên vị trí sample. Bit ẩn có thể chỉ là thay đổi nhỏ như +2 hoặc -2 tại một số vị trí cụ thể. Sau resampling, những thay đổi nhỏ này có thể bị mất, bị đổi dấu hoặc bị lan sang vị trí khác. Khi chương trình recover vẫn đọc theo các vị trí do khóa sinh ra, giá trị ở các vị trí đó không còn mang bit ban đầu nữa, nên thông điệp bị hỏng.

Bài lab cũng yêu cầu kiểm tra chất lượng định dạng sau resampling. Nếu recover thất bại nhưng file WAV bị sai định dạng hoặc bị cắt ngắn, kết luận sẽ không có ý nghĩa. Do đó cần chứng minh rằng file sau resampling vẫn là WAV hợp lệ, cùng số kênh, cùng độ sâu bit, cùng sample rate và cùng số frame với file tham chiếu.

1.2 Mục tiêu bài lab

- Đọc tham số bài lab và kiểm tra metadata của file âm thanh.
- Khôi phục thông điệp từ file stego ban đầu để chứng minh dữ liệu ẩn tồn tại.
- Dùng sox để resample file xuống sample rate thấp hơn và đưa lại về sample rate ban đầu.
- Kiểm tra file sau resampling vẫn là WAV hợp lệ.
- Thử khôi phục lại thông điệp sau resampling và chứng minh thông điệp bị hỏng.
- Hiểu vì sao resampling ảnh hưởng mạnh đến FHSS dựa trên vị trí sample.

2. Lab Environment

Thành phần	Ý nghĩa
cover.wav	File âm thanh tham chiếu ban đầu, dùng để so sánh khi recover.
stego.wav	File âm thanh đã được nhúng thông điệp FHSS.
params.txt	File chứa các tham số cần dùng: key, frame size, msg length, original rate và downsample rate.
recover.py	Công cụ khôi phục thông điệp FHSS từ file audio.
quality_check.py	Kiểm tra file sau resampling vẫn có cấu trúc WAV hợp lệ.
resample break check.py	Kiểm tra thông điệp sau resampling đã bị hỏng hay chưa.

sox / soxi	Công cụ âm thanh dùng để resample và xem thông tin file WAV.
------------	--

3. Tham số dùng trong bài

Trong bản hướng dẫn chuẩn, các tham số được dùng trực tiếp như sau:

Tham số	Giá trị	Ý nghĩa
KEY	46820	Khóa sinh chuỗi vị trí sample cần đọc.
FRAME_SIZE	1024	Kích thước frame dùng để ánh xạ bit vào các vùng sample.
MSG_LEN	8	Độ dài thông điệp tính theo ký tự.
ORIGINAL_RATE	44100	Sample rate ban đầu của file WAV.
DOWNSAMPLE_RATE	22050	Sample rate thấp hơn dùng để resample.

4. Lab Tasks

Task 1 - Kiểm tra file và tham số

```
cd ~
ls -l
cat params.txt
soxi stego.wav
```

Bước này xác nhận các file cần thiết đã có trong thư mục làm việc và đọc các tham số chính của bài lab. Lệnh soxi cho biết stego.wav là file WAV mono, 16-bit, 44100 Hz. Đây là điều kiện để các bước recover và resample phía sau có ý nghĩa.

Kết quả cần chú ý: params.txt phải có KEY=46820, FRAME_SIZE=1024, MSG_LEN=8, ORIGINAL_RATE=44100 và DOWNSAMPLE_RATE=22050.

Task 2 - Khôi phục thông điệp từ file stego ban đầu

```
./recover.py --cover cover.wav --input stego.wav --output original_recovered.txt --log original_recover.log --key 46820 --frame-size 1024 -
-msg-len 8
cat original_recovered.txt
```

Bước này chứng minh file stego.wav thật sự chứa thông điệp FHSS và các tham số ban đầu là đúng. Chương trình đọc các vị trí sample do key 46820 sinh ra, mỗi frame có kích thước 1024 sample, và đọc 8 ký tự thông điệp.

Kết quả mong đợi là original_recovered.txt chứa RESAMPLE. Nếu bước này sai, không nên làm tiếp vì có thể key, frame size hoặc msg length đang sai.

Task 3 - Resample xuống sample rate thấp hơn

```
sox stego.wav downsampled.wav rate 22050
soxi downsampled.wav
```

Lệnh sox tạo downsampled.wav bằng cách đổi sample rate từ 44100 Hz xuống 22050 Hz. Đây không phải chỉ là đổi nhãn metadata; sox phải tính lại tín hiệu âm thanh trên lưới sample mới.

Kết quả soxi cần cho thấy downsampled.wav có Sample Rate là 22050. Bước này làm thay đổi cấu trúc sample thời gian, là nguyên nhân làm dữ liệu FHSS dễ bị hỏng.

Task 4 - Resample ngược về sample rate ban đầu

```
sox downsampled.wav resampled.wav rate 44100
mkdir -p findings
soxi resampled.wav | tee findings/resample_info.txt
```

Bước này đưa file đã downsample về lại 44100 Hz để nhìn bề ngoài giống file ban đầu hơn. Tuy nhiên, các sample bên trong đã được nội suy và làm tròn, nên các thay đổi nhỏ dùng để biểu diễn bit FHSS có thể không còn giữ nguyên.

File findings/resample_info.txt được tạo từ output thật của soxi và được checkwork dùng để xác nhận resampled.wav đã được tạo và đọc được.

Task 5 - Kiểm tra file sau resampling vẫn hợp lệ

```
./quality_check.py --reference cover.wav --input resampled.wav
```

Bước này kiểm tra resampled.wav vẫn là WAV hợp lệ, có cùng số kênh, cùng sample width, cùng sample rate và cùng số frame với cover.wav. Nếu QUALITY_OK xuất hiện, có nghĩa là file không bị hỏng định dạng.

Ý nghĩa của bước này là tách biệt hai vấn đề: recover thất bại không phải vì file hỏng, mà vì dữ liệu FHSS bên trong đã bị biến dạng sau resampling.

Task 6 - Thử recover sau resampling

```
./recover.py --cover cover.wav --input resampled.wav --output resampled_recovered.txt --log resampled_recover.log --key 46820 --frame-size 1024 --msg-len 8
cat resampled_recovered.txt
```

Bước này dùng lại đúng key, frame size và msg length như lúc recover file gốc. Nếu thông điệp không còn là RESAMPLE, điều đó cho thấy resampling đã làm mất hoặc làm sai các bit FHSS.

Task 7 - Kiểm tra thông điệp đã bị hỏng

```
./resample_break_check.py
```

Script này đọc resampled_recovered.txt. Nếu nội dung khác RESAMPLE, nó in RESAMPLE_BROKEN_OK. Đây là bằng chứng checkwork rằng resampling đã phá thông điệp ẩn.

5. Checkwork

Sau khi hoàn thành các task, quay về terminal host và chạy:

```
checkwork
```